Pontificia Universidad Católica Madre y Maestra.

Presentado por:

José Ariel Martínez Dabas

Matricula:

1014-4270

Asignatura:

Programación Web Avanzada

Dirigido a:

Carlos Camacho

Título:

Práctica de Docker Básico

Link de Docker Hub:

https://hub.docker.com/r/josedabas/practica-icc354/tags

Práctica de Docker Básico

Introducción

El objetivo principal reside en la creación de una imagen de contenedor optimizada, capaz de ejecutarse de manera aislada en cualquier sistema que soporte el motor de Docker, eliminando la dependencia de configuraciones locales específicas. Para lograr esto, se ha diseñado un flujo de trabajo que abarca desde la escritura de un Dockerfile eficiente hasta la publicación del artefacto final en el registro público Docker Hub.

Desarrollo

Estrategia de Construcción (Dockerfile)

Para cumplir con el requisito de Multi-stage build, diseñamos un Dockerfile dividido en dos etapas claras. Esto nos permitió compilar el proyecto que utiliza Gradle y gRPC en una imagen robusta, pero ejecutarlo en una imagen.

Etapa 1 (Builder): Se utilizó eclipse-temurin:17-jdk-jammy para garantizar la compatibilidad con las librerías nativas de gRPC que requieren glibc.

Etapa 2 (Runtime): Se utilizó eclipse-temurin:17-jre-alpine, reduciendo drásticamente el tamaño final de la imagen al eliminar el código fuente y las herramientas de construcción.

Código del Dockerfile:

```
FROM eclipse-temurin:17-jdk-jammy AS builder

WORKDIR /home/gradle/src

COPY gradlew .
COPY gradle gradle
COPY app app
COPY settings.gradle .

RUN chmod +x gradlew

RUN ./gradlew :app:shadowJar --no-daemon

FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

RUN mkdir /logs
VOLUME /logs

COPY --from=builder /home/gradle/src/app/build/libs/*.jar app.jar

ENV SERVER_PORT=7000

EXPOSE ${SERVER_PORT}

ENTRYPOINT ["java", "-Dorg.slf4j.simpleLogger.logFile=/logs/app.log", "-jar", "app.jar"]
```

Manejo de Variables de Entorno y Puertos

Para cumplir con el requerimiento de cambiar el puerto de arranque dinámicamente, se modificó la clase Main.java para leer la variable de entorno SERVER_PORT.

```java
// Leemos la variable de entorno de Docker. Si no existe, usamos 7000 por
// defecto.
String strPort = System.getenv("SERVER_PORT");
int port = (strPort != null) ? Integer.parseInt(strPort) : 7000;

// Iniciar el servidor Javalin
Javalin app = Javalin.create(config -> {

    // Configuración de archivos estáticos para las plantillas
    config.staticFiles.add(staticFileConfig -> {
        staticFileConfig.hostedPath = "/";
        staticFileConfig.directory = "public/templates";
        staticFileConfig.location = Location.CLASSPATH;
        staticFileConfig.precompress = false;
        staticFileConfig.aliasCheck = null;
    });

    // Configuración de archivos estáticos para la carpeta pública
    config.staticFiles.add(staticFileConfig -> {
        staticFileConfig.hostedPath = "/";
        staticFileConfig.directory = "public";
        staticFileConfig.location = Location.CLASSPATH;
        staticFileConfig.precompress = false;
        staticFileConfig.aliasCheck = null;
    });

}).start(port);
```

Durante la ejecución del contenedor, utilizamos el flag -e para sobrescribir este valor, demostrando flexibilidad en el despliegue: docker run -e SERVER_PORT=8000

Conectividad con Base de Datos (MongoDB)

Uno de los retos principales fue la conexión a la base de datos. Al estar MongoDB alojado en el sistema operativo Windows y no dentro del contenedor, la cadena de conexión localhost fallaba.

Solución implementada:

- Se configuró la aplicación para leer la URI de conexión mediante variables de entorno (DB_URI)
- Se utilizó el alias de red especial de Docker: host.docker.internal

Comando final de ejecución:

```
PS C:\Users\josea\OneDrive\Desktop\Programacion Web\Proyecto Final-2> docker run -d -p 8080:8000 -e SERVER_PORT=8000 -v ${PWD}/mis-log
s:/logs --name mi-app-web josedabas/practica-icc354:v2
docker: Error response from daemon: Conflict. The container name "/mi-app-web" is already in use by container "ae53fdba986a9380f2cc31d
a4ef6b282f97c854a69be0f35e71e856d8d40c458". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
PS C:\Users\josea\OneDrive\Desktop\Programacion Web\Proyecto Final-2> docker run -d `
>>   -p 8080:8000 `
>>   -e SERVER_PORT=8000 `
>>   -e DB_URI="mongodb://host.docker.internal:27017" `
>>   -v ${PWD}/mis-logs:/logs `
>>   --name mi-app-web-v2 `
>>   josedabas/practica-icc354:v2
2345f4135ba42da6e5f02666cba59e95a5226d6a76698613a726b8cf016a772b
PS C:\Users\josea\OneDrive\Desktop\Programacion Web\Proyecto Final-2>
```

Evidencias de Funcionamiento

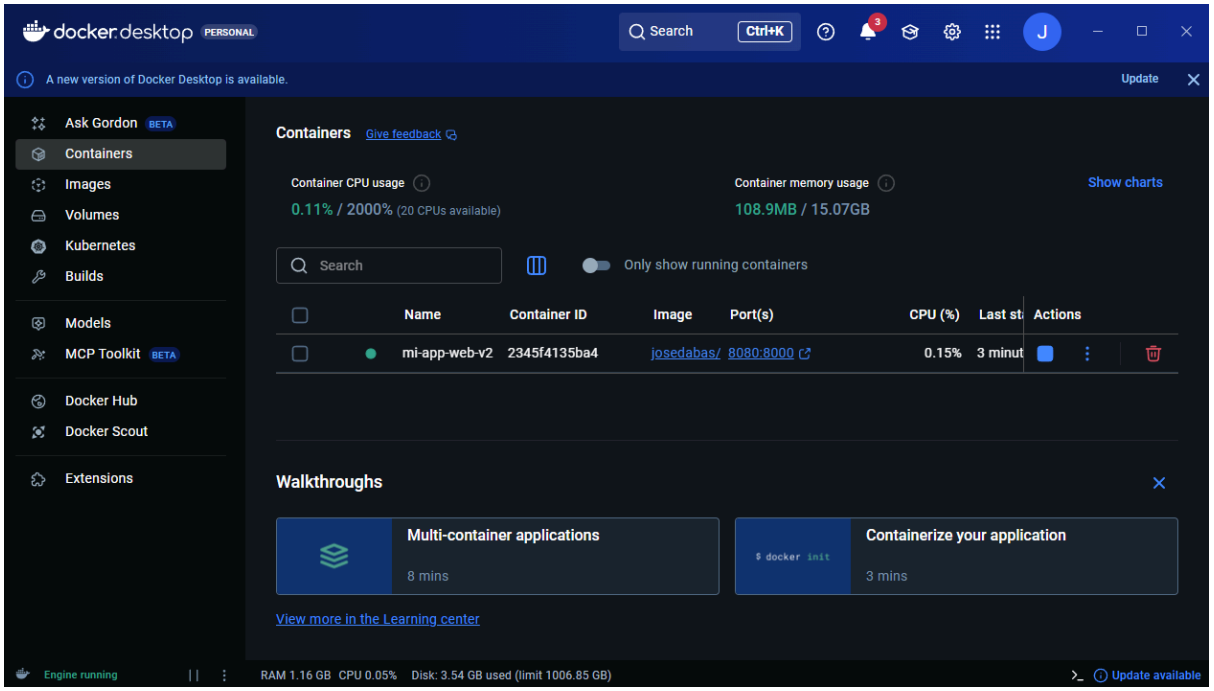
Construcción de la imagen

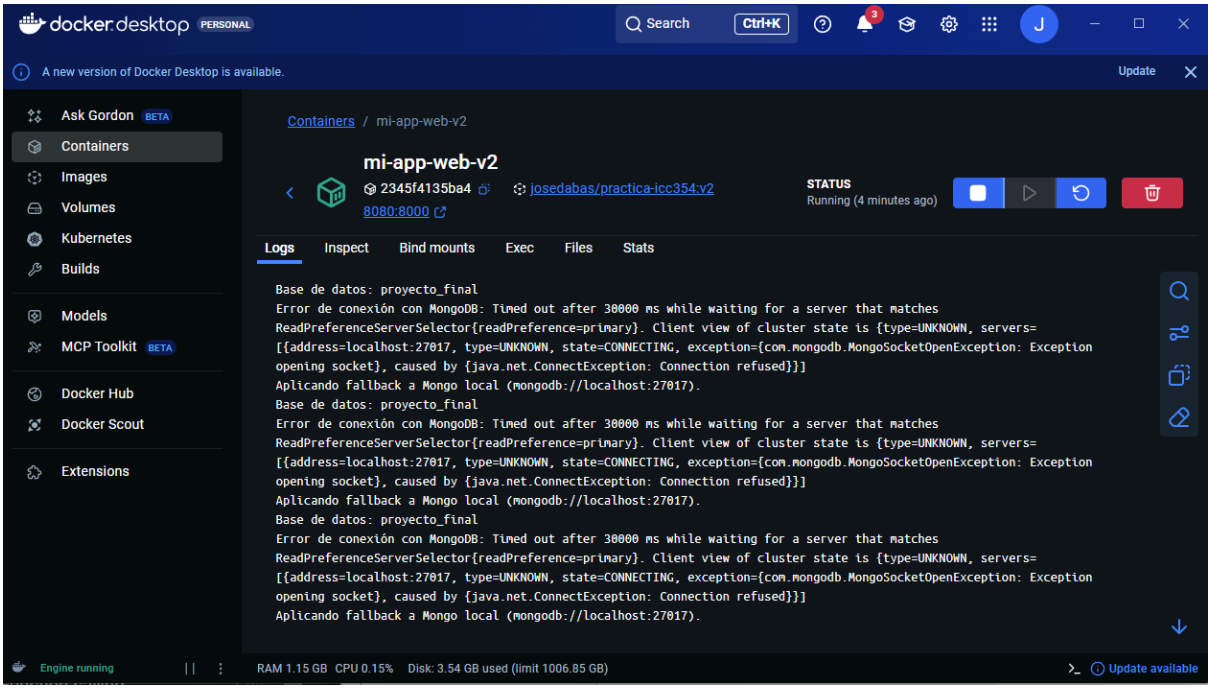
```
docker build -t josedabas/practica-icc354:v2 . Web\Proyecto Final-2>
[+] Building 64.9s (19/19) FINISHED                                                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 554B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jre-alpine                                             0.8s
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk-jammy                                              0.8s
 => [auth] library/eclipse-temurin:pull token for registry-1.docker.io                                                       0.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 97B                                                                                              0.0s
 => [builder 1/8] FROM docker.io/library/eclipse-temurin:17-jdk-jammy@sha256:2d05c919f87a3ebcd4853d595f323f75b05f545bf61bf3ad81  0.0s
 => => resolve docker.io/library/eclipse-temurin:17-jdk-jammy@sha256:2d05c919f87a3ebcd4853d595f323f75b05f545bf61bf3ad811cdfaa1d  0.0s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 13.77kB                                                                                          0.0s
 => [stage-1 1/4] FROM docker.io/library/eclipse-temurin:17-jre-alpine@sha256:7aa804a1824d18d06c68598fe1c2953b5b203823731be7b92  0.0s
 => => resolve docker.io/library/eclipse-temurin:17-jre-alpine@sha256:7aa804a1824d18d06c68598fe1c2953b5b203823731be7b9298bb3e0f  0.0s
 => CACHED [builder 2/8] WORKDIR /home/gradle/src                                                                             0.0s
 => CACHED [builder 3/8] COPY gradlew .                                                                                       0.0s
 => CACHED [builder 4/8] COPY gradle gradle                                                                                   0.0s
 => [builder 5/8] COPY app app                                                                                                0.0s
 => [builder 6/8] COPY settings.gradle .                                                                                      0.0s
 => [builder 7/8] RUN chmod +x gradlew                                                                                        0.2s
 => [builder 8/8] RUN ./gradlew :app:shadowJar --no-daemon                                                                   62.2s
 => CACHED [stage-1 2/4] WORKDIR /app                                                                                         0.0s
                                0.0s
 => => exporting config sha256:ccdc91a38aea77b802892c84c80c9ed478572cfa6979f48936e90947828cc89d                               0.0s

 => => exporting attestation manifest sha256:39018d8d3167365fa4169f62d77a035976241aabd2c9a646922e25e694976e12                 0.0s

                                0.0s
 => => exporting config sha256:ccdc91a38aea77b802892c84c80c9ed478572cfa6979f48936e90947828cc89d                               0.0s
 => => exporting attestation manifest sha256:39018d8d3167365fa4169f62d77a035976241aabd2c9a646922e25e694976e12                 0.0s
                                0.0s
                                0.0s
 => => exporting config sha256:ccdc91a38aea77b802892c84c80c9ed478572cfa6979f48936e90947828cc89d                               0.0s
                                0.0s
                                0.0s
 => => exporting config sha256:ccdc91a38aea77b802892c84c80c9ed478572cfa6979f48936e90947828cc89d                               0.0s
 => => exporting attestation manifest sha256:39018d8d3167365fa4169f62d77a035976241aabd2c9a646922e25e694976e12                 0.0s
 => => exporting manifest list sha256:1dc7c1d24f3d45733797195ee154b793e7fc0193559c5f93b61954c6d7c77106                         0.0s
 => => naming to docker.io/josedabas/practica-icc354:v2                                                                       0.0s
 => => exporting config sha256:ccdc91a38aea77b802892c84c80c9ed478572cfa6979f48936e90947828cc89d                               0.0s
 => => exporting attestation manifest sha256:39018d8d3167365fa4169f62d77a035976241aabd2c9a646922e25e694976e12                 0.0s
 => => exporting manifest list sha256:1dc7c1d24f3d45733797195ee154b793e7fc0193559c5f93b61954c6d7c77106                         0.0s
 => => naming to docker.io/josedabas/practica-icc354:v2                                                                       0.0s
 => => exporting attestation manifest sha256:39018d8d3167365fa4169f62d77a035976241aabd2c9a646922e25e694976e12                 0.0s
 => => exporting manifest list sha256:1dc7c1d24f3d45733797195ee154b793e7fc0193559c5f93b61954c6d7c77106                         0.0s
 => => naming to docker.io/josedabas/practica-icc354:v2                                                                       0.0s
 => => exporting manifest list sha256:1dc7c1d24f3d45733797195ee154b793e7fc0193559c5f93b61954c6d7c77106                         0.0s
 => => naming to docker.io/josedabas/practica-icc354:v2                                                                       0.0s
 => => naming to docker.io/josedabas/practica-icc354:v2                                                                       0.0s
```

Logs persistentes

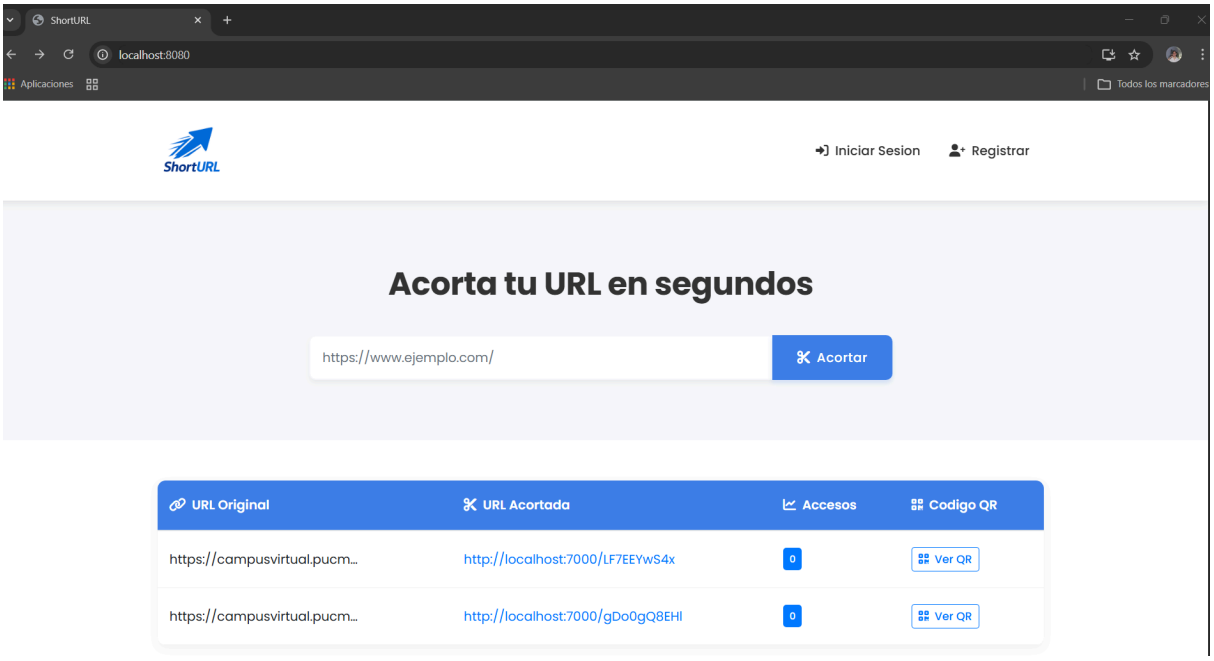
```
[main] INFO org.mongodb.driver.client - MongoClient with metadata {"driver": {"name": "mongo-java-driver|sync", "version": "4.11.1"}, "os": {"type": "Linux", "name": "Linux", "architecture": "amd64", "version":
"6.6.87.2-microsoft-standard-WSL2"}, "platform": "Java/Eclipse Adoptium/17.0.18+8"} created with settings MongoClientSettings{readPreference=primary, writeConcern=WriteConcern{w=null, wTimeout=null ms,
journal=null}, retryWrites=true, retryReads=true, readConcern=ReadConcern{level=null}, credential=null, transportSettings=null, streamFactoryFactory=null, commandListeners=[],
codecRegistry=ProvidersCodecRegistry{codecProviders=[ValueCodecProvider{}, BsonValueCodecProvider{}, DBRefCodecProvider{}, DBObjectCodecProvider{}, DocumentCodecProvider{}, CollectionCodecProvider{},
IterableCodecProvider{}, MapCodecProvider{}, GeoJsonCodecProvider{}, GridFSFileCodecProvider{}, Jsr310CodecProvider{}, JsonObjectCodecProvider{}, BsonCodecProvider{}, EnumCodecProvider{},
com.mongodb.client.model.mql.ExpressionCodecProvider@52525845, com.mongodb.Jep395RecordCodecProvider@3b94d659, com.mongodb.KotlinCodecProvider@24b1d79b]}, loggerSettings=LoggerSettings{maxDocumentLength=1000},
clusterSettings={hosts=[localhost:27017], srvServiceName=mongodb, mode=SINGLE, requiredClusterType=UNKNOWN, requiredReplicaSetName='null', serverSelector='null', clusterListeners='[]',
serverSelectionTimeout='30000 ms', localThreshold='15 ms'}, socketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=0, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null,
username=null, password=null}}, heartbeatSocketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=10000, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null, username=null,
password=null}}, connectionPoolSettings=ConnectionPoolSettings{maxSize=100, minSize=0, maxWaitTimeMS=120000, maxConnectionLifeTimeMS=0, maxConnectionIdleTimeMS=0, maintenanceInitialDelayMS=0,
maintenanceFrequencyMS=60000, connectionPoolListeners=[], maxConnecting=2}, serverSettings=ServerSettings{heartbeatFrequencyMS=10000, minHeartbeatFrequencyMS=500, serverListeners='[]',
serverMonitorListeners='[]'}, sslSettings=SslSettings{enabled=false, invalidHostNameAllowed=false, context=null}, applicationName='null', compressorList=[], uuidRepresentation=UNSPECIFIED, serverApi=null,
autoEncryptionSettings=null, dnsClient=null, inetAddressResolver=null, contextProvider=null}
[cluster-ClusterId{value='69923f04c583d27858c4aa8c', description='null'}-localhost:27017] INFO org.mongodb.driver.cluster - Exception in monitor thread while connecting to server localhost:27017
com.mongodb.MongoSocketOpenException: Exception opening socket
        at com.mongodb.internal.connection.SocketStream.lambda$open$0(SocketStream.java:84)
        at java.base/java.util.Optional.orElseThrow(Unknown Source)
        at com.mongodb.internal.connection.SocketStream.open(SocketStream.java:84)
        at com.mongodb.internal.connection.InternalStreamConnection.open(InternalStreamConnection.java:211)
        at com.mongodb.internal.connection.DefaultServerMonitor$ServerMonitorRunnable.lookupServerDescription(DefaultServerMonitor.java:196)
        at com.mongodb.internal.connection.DefaultServerMonitor$ServerMonitorRunnable.run(DefaultServerMonitor.java:156)
        at java.base/java.lang.Thread.run(Unknown Source)
Caused by: java.net.ConnectException: Connection refused
        at java.base/sun.nio.ch.Net.pollConnect(Native Method)
        at java.base/sun.nio.ch.Net.pollConnectNow(Unknown Source)
        at java.base/sun.nio.ch.NioSocketImpl.timedFinishConnect(Unknown Source)
        at java.base/sun.nio.ch.NioSocketImpl.connect(Unknown Source)
        at java.base/java.net.SocksSocketImpl.connect(Unknown Source)
        at java.base/java.net.Socket.connect(Unknown Source)
        at com.mongodb.internal.connection.SocketStreamHelper.initialize(SocketStreamHelper.java:76)
        at com.mongodb.internal.connection.SocketStream.initializeSocket(SocketStream.java:104)
        at com.mongodb.internal.connection.SocketStream.open(SocketStream.java:78)
        ... 4 more
[main] INFO org.mongodb.driver.cluster - No server chosen by ReadPreferenceServerSelector{readPreference=primary} from cluster description ClusterDescription{type=UNKNOWN, connectionMode=SINGLE,
serverDescriptions=[ServerDescription{address=localhost:27017, type=UNKNOWN, state=CONNECTING, exception={com.mongodb.MongoSocketOpenException: Exception opening socket}, caused by {java.net.ConnectException:
Connection refused}}]}. Waiting for 30000 ms before timing out
[main] INFO org.mongodb.driver.client - MongoClient with metadata {"driver": {"name": "mongo-java-driver|sync", "version": "4.11.1"}, "os": {"type": "Linux", "name": "Linux", "architecture": "amd64", "version":
"6.6.87.2-microsoft-standard-WSL2"}, "platform": "Java/Eclipse Adoptium/17.0.18+8"} created with settings MongoClientSettings{readPreference=primary, writeConcern=WriteConcern{w=null, wTimeout=null ms,
journal=null}, retryWrites=true, retryReads=true, readConcern=ReadConcern{level=null}, credential=null, transportSettings=null, streamFactoryFactory=null, commandListeners=[],
codecRegistry=ProvidersCodecRegistry{codecProviders=[ValueCodecProvider{}, BsonValueCodecProvider{}, DBRefCodecProvider{}, DBObjectCodecProvider{}, DocumentCodecProvider{}, CollectionCodecProvider{},
IterableCodecProvider{}, MapCodecProvider{}, GeoJsonCodecProvider{}, GridFSFileCodecProvider{}, Jsr310CodecProvider{}, JsonObjectCodecProvider{}, BsonCodecProvider{}, EnumCodecProvider{},
com.mongodb.client.model.mql.ExpressionCodecProvider@52525845, com.mongodb.Jep395RecordCodecProvider@3b94d659, com.mongodb.KotlinCodecProvider@24b1d79b]}, loggerSettings=LoggerSettings{maxDocumentLength=1000},
clusterSettings={hosts=[host.docker.internal:27017], srvServiceName=mongodb, mode=SINGLE, requiredClusterType=UNKNOWN, requiredReplicaSetName='null', serverSelector='null', clusterListeners='[]',
serverSelectionTimeout='30000 ms', localThreshold='15 ms'}, socketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=0, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null,
username=null, password=null}}, heartbeatSocketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=10000, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null, username=null,
password=null}}, connectionPoolSettings=ConnectionPoolSettings{maxSize=100, minSize=0, maxWaitTimeMS=120000, maxConnectionLifeTimeMS=0, maxConnectionIdleTimeMS=0, maintenanceInitialDelayMS=0,
maintenanceFrequencyMS=60000, connectionPoolListeners=[], maxConnecting=2}, serverSettings=ServerSettings{heartbeatFrequencyMS=10000, minHeartbeatFrequencyMS=500, serverListeners='[]',
serverMonitorListeners='[]'}, sslSettings=SslSettings{enabled=false, invalidHostNameAllowed=false, context=null}, applicationName='null', compressorList=[], uuidRepresentation=UNSPECIFIED, serverApi=null,
autoEncryptionSettings=null, dnsClient=null, inetAddressResolver=null, contextProvider=null}
[cluster-ClusterId{value='69923f22c583d27858c4aa8d', description='null'}-host.docker.internal:27017] INFO org.mongodb.driver.cluster - Monitor thread successfully connected to server with description
ServerDescription{address=host.docker.internal:27017, type=STANDALONE, state=CONNECTED, ok=true, minWireVersion=0, maxWireVersion=21, maxDocumentSize=16777216, logicalSessionTimeoutMinutes=30,
roundTripTimeNanos=18600742}
[main] INFO io.javalin.Javalin - Starting Javalin ...
[main] INFO org.eclipse.jetty.server.Server - jetty-11.0.15; built: 2023-04-11T18:37:53.775Z; git: 5bc5e562c8d05c5862505aebe5cf83a61bdbcb96; jvm 17.0.18+8
[main] INFO org.eclipse.jetty.server.session.DefaultSessionIdManager - Session workerName=node0
[main] INFO org.eclipse.jetty.server.handler.ContextHandler - Started i.j.j.@52066604{/,null,AVAILABLE}
[main] INFO org.eclipse.jetty.server.AbstractConnector - Started ServerConnector@60856961{HTTP/1.1, (http/1.1)}{0.0.0.0:8000}
[main] INFO org.eclipse.jetty.server.Server - Started Server@3d3e5463{STARTING}[11.0.15,sto=0] @30940ms
[main] INFO io.javalin.Javalin -
        __                  __                  ______
      / /___ __  ____ _/ (_)___     / ____/
 __  / / __ `/ | / / __ `/ / / __ \   /___ \
/ /_/ / /_/ /| |/ / /_/ / / / / / /  ____/ /
\____/\__,_/ |___/\__,_/_/_/_/ /_/  /_____/

      https://javalin.io/documentation

[main] INFO io.javalin.Javalin - Static file handler added: StaticFileConfig(hostedPath=/, directory=public/templates, location=CLASSPATH, precompress=false, aliasCheck=null, headers={Cache-Control=max-age=0},
skipFileFunction=Function1<jakarta.servlet.http.HttpServletRequest, java.lang.Boolean>, mimeTypes=io.javalin.http.staticfiles.MimeTypesConfig@2cc3ad05). File system location:
'jar:file:/app/app.jar!/public/templates'
```

Ejecución y Docker Hub





Aplicación Web en funcionamiento



Conclusión

La implementación de Docker en el proyecto ha permitido desacoplar completamente la aplicación de la infraestructura de desarrollo. A través del uso de construcciones en múltiples etapas, logramos reducir el tamaño de la imagen final significativamente al usar una base Alpine, sin sacrificar la capacidad de compilar dependencias complejas como gRPC en la etapa de construcción. Asimismo, la configuración de volúmenes y variables de entorno asegura que la imagen sea inmutable pero configurable, cumpliendo con los principios de las aplicaciones "Cloud Native". La aplicación es ahora capaz de ejecutarse en cualquier entorno que soporte Docker, manteniendo la persistencia de sus registros y conectándose flexiblemente a servicios de datos externos.